

Beginner's Guide to This Project

This document explains **every file and function** in this project in plain language. It's written for someone who is new to React, Express, or full-stack apps in general.

The app is a simple **Login + Dashboard** system:

1. A user types an email/password on a Login page.
 2. The browser (frontend) sends that to a server (backend).
 3. The server checks the database, and if correct, sends back a "token" and user info.
 4. The browser saves that info and shows a Dashboard page.
 5. The user can click Logout to clear everything and go back to Login.
-

1. The Big Picture (two separate projects)

```
ReactApp1/  
├── src/          <- FRONTEND (React) - runs in the browser  
└── backend/      <- BACKEND (Node.js/Express) - runs on a server
```

These are **two independent programs** that talk to each other over HTTP:

```
[ React app in browser ] --- HTTP requests ---> [ Express server ] ---> [ PostgreSQL database  
  (port 5173+)                               (port 5000)
```

You start them separately:

- Frontend: `npm run dev` (in the project root)
 - Backend: `npm run dev` (inside the `backend` folder)
-

2. Frontend (the `src` folder)

2.1 How the app boots up

`src/main.jsx` is the very first file that runs. Think of it as the "power switch."

```
createRoot(document.getElementById('root')).render(<App />)
```

It finds the empty `<div id="root">` in `index.html` and tells React: "put the whole app inside this div."

`src/App.jsx` is almost empty on purpose — it just renders `AppRoutes`, which decides what page to show based on the URL.

```
function App() {  
  return <AppRoutes />  
}
```

2.2 Routing — `src/routes/AppRoutes.jsx`

This file is like a **map** that says "if the URL is X, show component Y."

URL	What shows
<code>/</code>	Redirects straight to <code>/login</code>
<code>/login</code>	The <code>Login</code> page
<code>/dashboard</code>	The <code>Dashboard</code> page, but only if logged in

```
<Route path="/" element={<Navigate to="/login"/>} />  
<Route path="/login" element={<Login />} />  
<Route path="/dashboard" element={  
  <ProtectedRoute><Dashboard /></ProtectedRoute>  
} />
```

`<Navigate to="/login"/>` is React Router's way of saying "send the user to this URL immediately" without them clicking anything.

2.3 Guarding the Dashboard — `src/routes/ProtectedRoute.jsx`

This is a **gatekeeper component**. It wraps any page that requires login.

```
function ProtectRoute({ children }) {  
  const user = localStorage.getItem("user");  
  if (!user) {  
    return <Navigate to="/" />; // not logged in -> bounce to login  
  }  
  return children; // logged in -> show the real page  
}
```

`children` is a special React prop — it means "whatever was placed between the opening and closing tag." In `AppRoutes.jsx` we wrote:

```
<ProtectedRoute>  
  <Dashboard />  
</ProtectedRoute>
```

So `children` here is `<Dashboard />`. `ProtectedRoute` decides whether to actually render it.

`localStorage` is a small key-value storage built into every browser. It survives page refreshes (unlike React state), which is why it's used to "remember" that someone is logged in.

2.4 The Login page — `src/pages/login.jsx`

This is the **parent component** that owns the login form's data and logic. Beginner tip: in React, "state" is just data a component remembers and can change. `useState` is the hook that creates that memory.

```
const [email, setEmail] = useState(""); // remembers what's typed in the email box
const [password, setPassword] = useState(""); // same for password
const [error, setError] = useState(""); // remembers an error message, if any
const navigate = useNavigate(); // lets us change the URL in code
```

`handleLogin` is the function that runs when the Login button is clicked:

```
const handleLogin = async () => {
  try {
    const result = await loginUser({ email, password }); // 1. ask the backend
    setError("");
    if (result.success) {
      localStorage.setItem("user", JSON.stringify(result.user)); // 2. remember the user
      localStorage.setItem("token", result.token); // 3. remember the token
      navigate("/dashboard"); // 4. go to dashboard
    }
  } catch (error) {
    setError(error.response?.data?.message || "login failed"); // show error text
  }
};
```

Step by step:

1. Calls the backend's `/login` endpoint through `loginUser(...)`.
2. If the backend says `success: true`, it saves the user's info as a **string** (because `localStorage` can only store text — that's what `JSON.stringify` is for).
3. Saves the **JWT token** too (explained in the backend section).
4. Sends the browser to `/dashboard`.

If anything goes wrong (wrong password, server down, etc.), the `catch` block grabs the error message and displays it instead of crashing.

The JSX at the bottom just lays out the page and conditionally shows the error message:

```
{ error && <p className="text-red-500 text-center mt-2">{error}</p> }
```

This reads as: "if `error` is a non-empty string, render this paragraph." It's a common React pattern called **conditional rendering**.

2.5 The reusable form — `src/features/authentication/Components/LoginForm.jsx`

This component doesn't manage any state itself — it receives everything as **props** (inputs) from `login.jsx` and just displays the form:

```
<Input label="Email" value={email} onChange={(e) => setEmail(e.target.value)} ... />
<Input label="Password" value={password} onChange={(e) => setPassword(e.target.value)} ... />
<Button text="Login" onClick={onLogin} type="button" />
```

This "parent owns the data, child just displays it" pattern is called **controlled components** — very common in React forms.

2.6 Reusable building blocks

`src/components/inputs/Input.jsx` — a styled `<input>` with a label, reused for both the email and password fields instead of writing the same markup twice.

`src/components/Buttons/Button.jsx` — a styled `<button>` reused anywhere a button is needed, so all buttons in the app look consistent.

Why bother with these? If you ever want to change how every input/button looks, you edit **one file** instead of hunting through the whole app.

2.7 Talking to the backend

`src/services/apiClient.jsx` sets up `axios` (a library for making HTTP requests) once, with the backend's address baked in:

```
const apiClient = axios.create({
  baseUrl: "http://localhost:5000/api",
  headers: { "Content-Type": "application/json" }
});
```

Every other file uses this `apiClient` instead of repeating the URL everywhere.

`src/features/authentication/services/authService.jsx` wraps the actual login request:

```
export const loginUser = async (data) => {
  const response = await apiClient.post("/auth/login", data);
  return response.data;
};
```

`data` here is `{ email, password }`. This function's only job is "send this data to `/auth/login` and give me back the server's reply."

2.8 The Dashboard — `src/pages/dashBoard.jsx`

```
const stored = localStorage.getItem("user");
let user;
try {
  user = stored ? JSON.parse(stored) : null;
} catch {
  user = null;
}
```

This reads the saved user back out of `localStorage`. Since it was saved as text (`JSON.stringify`), it needs to be turned back into an object (`JSON.parse`). The `try/catch` protects against a crash if that text is ever broken or missing — it just falls back to `null` instead of breaking the whole page.

```
<h1>Welcome {user?.username}</h1>
```

The `?.` is called **optional chaining**. It means "if `user` exists, read `.username`; if `user` is `null`, just give me `undefined` instead of crashing."

Logout:

```
const logout = () => {
  localStorage.removeItem("user");
  navigate('/');
};
```

This deletes the saved user and sends the browser back to `/`, which (per the routing table) redirects to `/login`.

2.9 Pieces that exist but aren't wired in yet

- `useLogin.jsx` (a custom hook) and `authStore.jsx` (a Zustand global store) were built as an alternative way to manage login state/loading/errors, but `login.jsx` currently does this itself with plain `useState`. They're not bugs — just unused groundwork for a future refactor (e.g. sharing login state across multiple components).

3. Backend (the `backend` folder)

3.1 The entry point — `backend/src/app.js`

This file starts the actual web server.

```
const app = express();
app.use(cors()); // allow the React app (different port) to call this API
app.use(express.json()); // automatically parse incoming JSON request bodies

app.use("/api/auth", authRoutes); // anything starting with /api/auth goes here
app.use("/api/users", userRoutes); // anything starting with /api/users goes here
app.use("/api-docs", swaggerui.serve, swaggerui.setup(swaggerSpec)); // live API docs page

app.listen(5000, () => console.log("Server running on port 5000"));
```

Think of `app.use("/api/auth", authRoutes)` as: "if the request path starts with `/api/auth`, hand it off to the `authRoutes` file to figure out the rest."

3.2 Routes — "what URL maps to what function"

`backend/src/routes/authRoutes.js`

```
router.post("/login", login);
router.post("/register", register);
```

This says: "a `POST` request to `/login` should run the `login` function from the controller; `/register` runs `register`."

(The big comment blocks above each route starting with `/**` and `@swagger` are not code — they're documentation that auto-generates the page at `/api-docs`.)

`backend/src/routes/userRoutes.js`

```
router.get("/profile", verifyToken, (req, res) => {
  res.status(200).json({ success: true, user: req.user });
});
```

This route returns "whoever's token this is." Notice it has **two** functions: `verifyToken` runs first (it's middleware, explained next), and only if that passes does the inline function run.

3.3 Controllers — "the actual logic"

`backend/src/controller/authController.js` has two functions.

`login`:

```
const login = async (req, res) => {
  const { email, password } = req.body; // 1. read what the user sent
  const result = await pool.query( // 2. look the user up in the DB
    "SELECT * FROM users WHERE email=$1", [email]
  );
  if (result.rows.length === 0) {
```

```

    return res.status(400).json({ success: false, message: "user not found" });
  }
  const user = result.rows[0];
  const isValidPassword = await bcrypt.compare(password, user.password); // 3. check password
  if (!isValidPassword) {
    return res.status(401).json({ success: false, message: "Invalid Password" });
  }
  const token = jwt.sign(
    { id: user.id, email: user.email, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "1h" }
  ); // 4. issue a login token
  return res.status(200).json({ success: true, message: "Login successful", token, user: {...} });
};

```

Plain-English walkthrough:

1. Pull `email` / `password` out of the request body (what the frontend sent).
2. Run a SQL query: "find a row in the `users` table where the email matches." `$1` is a placeholder — the database safely substitutes `email` in, which also protects against SQL injection attacks.
3. **Passwords are never stored as plain text.** `bcrypt.compare` checks the typed password against the scrambled ("hashed") version stored in the database.
4. If everything checks out, `jwt.sign(...)` creates a **JWT (JSON Web Token)** — a signed, tamper-proof string that says "this is user #5, role User, valid for 1 hour." The frontend stores this token and can send it back later to prove who it is, without logging in again every time.

`register` :

```

const register = async (req, res) => {
  const { username, email, password } = req.body;
  const existingUser = await pool.query("SELECT * FROM users WHERE email=$1", [email]);
  if (existingUser.rows.length > 0) {
    return res.status(400).json({ success: false, message: "Email Already Exists" });
  }
  const hashedPassword = await bcrypt.hash(password, 10);
  await pool.query(
    "INSERT INTO users(username,email,password) VALUES($1,$2,$3)",
    [username, email, hashedPassword]
  );
  return res.status(201).json({ success: true, message: "User Registration Sucessfully" });
};

```

This creates a brand-new account: check the email isn't taken, **scramble the password with `bcrypt.hash`** (so even the database admin can't read it), then insert the new row.

3.4 Middleware — `backend/src/middleware/authMiddleware.js`

Middleware is a function that runs **before** the actual route handler, usually to check something.

```
const verifyToken = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader) {
    return res.status(401).json({ success: false, message: "Access Denied" });
  }
  const token = authHeader.split(" ")[1]; // header looks like "Bearer <token>"
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // attach the decoded info so later code can use it
    next(); // continue to the actual route
  } catch {
    return res.status(401).json({ success: false, message: "Invalid Token" });
  }
};
```

`next()` is the key concept: calling it says "I'm done checking, let the request continue to whatever it was actually trying to reach." If you don't call `next()`, the request stops here forever.

3.5 Database connection — `backend/src/config/db.js`

```
const pool = new Pool({
  user: "postgres",
  host: "localhost",
  database: "react_enterprise_db",
  password: "12345",
  port: 5432
});
module.exports = pool;
```

A `Pool` manages a small set of reusable connections to PostgreSQL instead of opening a brand-new connection for every query (which would be slow). Every controller imports this same `pool` and calls `.query(...)` on it.

Note for later: the password here is hardcoded directly in the file. In a real project this should come from a `.env` file (using the `dotenv` package, which is already installed) so secrets aren't committed to git.

3.6 API documentation — `backend/src/swagger.js`

This file doesn't affect how the app runs — it generates a web page (at `http://localhost:5000/api-docs`) listing every API endpoint, what data it expects, and what it returns, based on the `@swagger` comments in `authRoutes.js`. Useful for testing the API without writing frontend code.

4. The Full Login Journey, Start to Finish

1. User opens the site → redirected to `/login` (`AppRoutes.jsx`).
 2. User types email/password → stored in `login.jsx` 's `useState` .
 3. User clicks the `Button` inside `LoginForm` → calls `onLogin` → which is `handleLogin` in `login.jsx` .
 4. `handleLogin` calls `loginUser()` → `authService.jsx` → `apiClient.jsx` sends a `POST` request to `http://localhost:5000/api/auth/login` .
 5. On the backend: `app.js` routes it to `authRoutes.js` → which calls `login` in `authController.js` .
 6. `login` checks the database, verifies the password with `bcrypt` , and if valid, signs a JWT with `jsonwebtoken` and sends back `{ success, token, user }` .
 7. Back in the browser, `login.jsx` saves `user` and `token` to `localStorage` and navigates to `/dashboard` .
 8. `ProtectedRoute.jsx` sees there's a `user` in `localStorage` and allows `Dashboard` to render.
 9. `Dashboard` reads the saved `user` and displays the welcome message.
 10. Clicking **Logout** clears `localStorage` and sends the user back to `/login` .
-

5. Key Concepts Glossary

Term	Meaning
Component	A reusable piece of UI written as a JS function returning JSX.
Props	Data passed <i>into</i> a component from its parent (read-only).
State (<code>useState</code>)	Data a component remembers and can change over time.
Hook	A special function (starts with <code>use</code>) that plugs into React features like state or routing.
JSX	HTML-like syntax inside JavaScript, compiled into real DOM elements.
<code>localStorage</code>	Browser storage that survives page refreshes/closing the tab.
API / Endpoint	A URL on the server that does something specific (e.g. <code>/api/auth/login</code>).
Middleware	Code that runs in between a request arriving and the final handler responding.
JWT (token)	A signed string proving "who you are," sent with future requests instead of logging in again.
bcrypt hash	A one-way scrambled version of a password, safe to store in a database.
SQL query / <code>pool.query</code>	A command sent to the PostgreSQL database to read or write data.